BrainAI

ProSuite Industrial

KERNEL & USERSPACE ENCRYPTION

PRODUCT DOCUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 PUBLIC CLASSIFICATION
Core Architecture & Development	**ProED Engine CLI Reference**	Access Level: Public / Beta

---

Revision History

Version	Date	Author	Status	Description
1.0.0	2025-01-01	S. Köhne	Stable	Initial Command Line Interface
2.0.0	2026-02-26	S. Köhne	Stable	Hybrid Ring-0 Edition (ProKM Kernel Support & Armored Envelope)

1. Product Overview

The ProED Executable (`proed.exe` / `proed`) is the high-performance Command-Line Interface (CLI) of the ProED Core Engine. It enables the seamless encryption and decryption of files and data streams directly via the terminal or through automation scripts.

Core Features

- **Hybrid Acceleration:** The executable automatically detects whether the `ProKM` kernel driver is installed on the host system. If so, large files (e.g., backups or log archives) are processed directly via Ring-0; otherwise, the native user-space fallback takes over.
- **Stream Processing:** Files are processed in chunks in a highly memory-efficient manner. Even terabyte-sized database dumps can be securely encrypted on systems with minimal RAM.
- **Armored Envelope (AEAD):** The CLI automatically wraps encrypted files with the 48-byte header (including IV and MAC). During decryption, the file is strictly rejected if even a single bit has been altered.
- **Automation-Friendly:** Strict POSIX-compliant exit codes (return values) allow for reliable evaluation in CI/CD pipelines and shell scripts.

2. Technical Specifications

2.1 Global Constants

The executable strictly adheres to the limits and specifications of the Core Engine:

- **Envelope Header:** *48* Bytes (Written to the beginning of the file during encryption).
- **Key Size:** Exactly *32* Bytes (*256*-bit). Can be passed as a raw binary file (`-k key.bin`) or as a hex string (`--hexkey`).
- **Chunk Size (I/O):** 4 MB by default (Optimized for disk and kernel I/O).

2.2 Command-Line Parameters (CLI)

The executable expects the following syntax: `proed [MODE] [OPTIONS]`

- `-e, --encrypt` : Starts the encryption mode.
- `-d, --decrypt` : Starts the decryption mode.
- `-i, --input <file>` : Path to the input file.
- `-o, --output <file>`: Path to the output file.
- `-k, --key <file>` : Path to the 32-byte key file.
- `--hexkey <string>` : Alternatively: The 32-byte key as a 64-character hex string.

3. Global Error Management (Exit Codes)

For seamless integration into scripts, the ProED Executable uses standardized exit codes (return values). A value of `0` always indicates success.

Exit Code	***Meaning***	***Description***	***Criticality***
0	SUCCESS	Operation completed successfully. File was verified and written.	*None*
1	ERR_PARAM	Invalid CLI parameters (e.g., missing arguments, invalid key length).	*Medium (Logic error)*
2	ERR_AUTH	MAC comparison failed. The input file has been manipulated or is corrupted!	*High (Attack!)*
3	ERR_IO	Read/Write error (e.g., file not found, insufficient permissions).	*Medium (System error)*
4	ERR_KERNEL	The ProKM driver reported a critical hardware error.	*Fatal*

4. Linux / Bash Integration (Server & CI/CD)

On Linux and macOS, the executable is perfectly suited for cron jobs, backup scripts, or use within Docker containers.

4.1 Implementation Example (Backup Script)

```bash
#!/bin/bash

INPUT_FILE="/var/log/syslog.archive"
OUTPUT_FILE="/backups/syslog.archive.proed"
KEY_FILE="/etc/proed/master_key.bin"

echo "Starting secure encryption..."

# Call ProED CLI
./proed --encrypt -i "$INPUT_FILE" -o "$OUTPUT_FILE" -k "$KEY_FILE"

# Check exit code
EXIT_CODE=$?

if [ $EXIT_CODE -eq 0 ]; then
    echo "Backup successfully encrypted and secured."
    rm "$INPUT_FILE" # Delete original file
elif [ $EXIT_CODE -eq 3 ]; then
    echo "Error: Could not read or write file (I/O Error)."
else
    echo "Critical encryption error! (Code: $EXIT_CODE)"
    exit 1
fi
```

5. Windows PowerShell Integration (Industry & OT)

On Windows systems (especially in SCADA and PLC environments), `proed.exe` is frequently used to securely process configuration files or sensor dumps before they are transmitted over the network.

5.1 Implementation Example (Secure File Import)

```
$EncryptedFile = "C:\OT_Data\sensor_dump.proed"
$DecryptedFile = "C:\OT_Data\sensor_dump.csv"
$KeyHex = "A1B2C3D4E5F6..." # 64-character hex string

Write-Host "Verifying and decrypting sensor data..."

# Call ProED Executable and wait for completion
$process = Start-Process -FilePath "proed.exe" -ArgumentList "-d -i $

# Strict evaluation of the return code
if ($process.ExitCode -eq 0) {
    Write-Host "Integrity confirmed. File was successfully decrypted.
    # Start import into control software
}
elseif ($process.ExitCode -eq 2) {
    Write-Host "SECURITY ALERT: File was manipulated (MAC Error)!" -F
    # Trigger alarm, put facility into safe state
}
else {
    Write-Host "General system error (Code: $($process.ExitCode))." -
}
```

6. Python Subprocess Integration

If native C-bindings (`ctypes`) should not or cannot be used (e.g., in restrictive environments), the executable can be securely orchestrated directly from Python.

6.1 Implementation Example

```
import subprocess
import os

def decrypt_secure_file(input_path: str, output_path: str, key_path:
    """Decrypts a file via ProED CLI and checks for manipulation."""

    if not os.path.exists(input_path):
        raise FileNotFoundError(f"Input file {input_path} is missing.

    # Pass command securely as a list (prevents shell injection)
    cmd = [
        "./proed",
        "--decrypt",
        "-i", input_path,
        "-o", output_path,
        "-k", key_path
    ]

    try:
        # Executes the CLI and captures the exit code
        result = subprocess.run(cmd, capture_output=True, text=True,

        if result.returncode == 0:
            print("Success: File is authentic and decrypted.")
            return True
        elif result.returncode == 2:
            print("CRITICAL: Strict rejection! The file has been mani
            return False
        else:
            print(f"Engine Error (Code {result.returncode}): {result.
            return False

    except Exception as e:
```

```
        print(f"System error while calling the executable: {e}")
        return False
```

7. Node.js Child Process Integration

In JavaScript/Node.js backends, the executable can be invoked asynchronously via `child_process` to offload computationally intensive encryption or decryption of large files to a separate operating system process (preventing the main thread from blocking).

7.1 Implementation Example

```
const { spawn } = require('child_process');

function encryptFileAsync(inputPath, outputPath, hexKey) {
    return new Promise((resolve, reject) => {

        // Asynchronous invocation of the executable
        const proed = spawn('./proed', [
            '-e',
            '-i', inputPath,
            '-o', outputPath,
            '--hexkey', hexKey
        ]);

        proed.on('close', (code) => {
            if (code === 0) {
                console.log("Encryption via CLI completed successfull
                resolve(true);
            } else if (code === 3) {
                reject(new Error("I/O Error: Check file permissions."
            } else {
                reject(new Error(`Critical engine error. Exit code: $
            }
        });

        proed.on('error', (err) => {
            reject(new Error(`Error starting the executable: ${err.me
        });
    });
}
```

8. Best Practices & Security Guidelines

- **Key Passing:** In production scripts, always prefer passing the key as a file (`-k key.bin`) rather than via a hex string (`--hexkey`). Parameters passed via the command line (CLI) may temporarily be visible to other users of the system in the process list (e.g., via `htop` or Task Manager).
- **Secure Erasing (Wiping):** The executable automatically wipes the key and all sensitive data from its own RAM after processing. However, securely deleting the unencrypted original file from the hard drive is the responsibility of the calling script (e.g., using tools like `shred` or `sdelete`).
- **Kernel Fallback:** The executable detects the ProKM driver fully automatically. No separate CLI switches are required. However, ensure that the executing user (or service account) has the necessary permissions to communicate with `\\.\ProKM` (Windows) or `/dev/prokm` (Linux).

© 2026 Sascha Köhne	✉️ **Contact:** koehne83@googlemail.com	🛡️ **ProTU-Verified**
CEO / System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**